

Meeting Summarizer (Unthinkable Company Project Task)

Local, Free-Stack AI-Powered Meeting Transcription & Summarization

Student Information

Name: Akshat Sinha
Registration Number: 22BCE2218
Program: BTech CSE Core

GitHub Repository

https://github.com/akshatsinha0/AkshatSinha_22BCE2218_MeetingSummarizer

Project Overview

Meeting Summarizer is a comprehensive AI-powered application that transcribes meeting audio files and generates intelligent summaries with key decisions and action items. The project runs entirely locally with zero-cost operation, leveraging open-source technologies.

Key Highlights

- **100% Local & Free:** No cloud dependencies, runs completely offline
- **Real-time Transcription:** Live audio streaming with WebSocket support
- **Speaker Diarization:** Automatic speaker identification and labeling
- **AI Summarization:** LLM-powered summaries with key decisions and action items
- **Modern UI:** Next.js frontend with dark/light themes and accessibility features
- **Advanced Features:** Transcript editing, bookmarks, compliance checking, calendar integration

Technology Stack

Backend

- **Framework:** FastAPI (Python)
- **ASR:** OpenAI Whisper (faster-whisper, large-v3-turbo model)
- **LLM:** Ollama (local LLM - gemma3:4b)
- **Diarization:** pyannote.audio
- **Database:** SQLite
- **Performance:** Rust extensions (PyO3) for audio processing

Frontend

- **Framework:** Next.js 15.5.4 (App Router)
- **Language:** TypeScript
- **Styling:** Inline CSS with theme support
- **Real-time:** WebSocket for live transcription

Implemented Features

Core Features

1. Audio input support (MP3, WAV, multiple formats)
2. Text transcript generation with Whisper
3. Speaker diarization with pyannote
4. AI-powered summary generation
5. Key decisions extraction
6. Action items generation
7. Async job processing with progress tracking
8. PDF and DOCX export

Advanced Features

9. **Real-time Transcription:** Live audio streaming with WebSocket
10. **Transcript Editing:** Manual corrections with version history
11. **Bookmarks:** Mark important moments with timestamps
12. **Keyboard Shortcuts:** Quick navigation (Ctrl+K, Ctrl+T, Ctrl+L)

13. **Theme Toggle:** Dark/Light mode with accessibility controls
14. **Batch Processing:** Upload and process multiple files
15. **Calendar Integration:** Google Calendar API integration
16. **Compliance Checking:** Automatic PII and sensitive data detection
17. **Custom Analysis:** Reanalyze transcripts with custom prompts
18. **Playback Controls:** Variable speed (0.5x to 2.0x)

Project Structure

```
MeetingSummarizer/
  backend/
    app/
      main.py                # FastAPI application
      calendar_integration.py # Google Calendar API
    rust_audio/              # Rust audio processing
    requirements.txt         # Python dependencies
  frontend/
    app/
      page.tsx               # Home page
      live/page.tsx          # Live transcription
      jobs/[id]/page.tsx     # Job details
      calendar/page.tsx      # Calendar integration
      settings/page.tsx      # User settings
    package.json             # Node dependencies
  storage/                   # Local file storage
  README.md                  # Documentation
  .env.example               # Environment template
```

Setup & Installation

Prerequisites

- Node.js (for frontend)
- Python 3.11+ (for backend)
- FFmpeg (for audio processing)
- Ollama (for local LLM)
- Rust (for audio extensions)

Installation Steps

1. Clone the repository
2. Create Python virtual environment: `python -m venv .venv`

3. Install backend dependencies: `pip install -r backend/requirements.txt`
4. Build Rust extensions: `maturin develop`
5. Install frontend dependencies: `cd frontend && npm install`
6. Copy `.env.example` to `.env` and configure
7. Start backend: `./backend/run_dev.ps1`
8. Start frontend: `cd frontend && npm run dev`

Submission Compliance

Checklist:

- Repository is public and accessible
- Branch name is `main`
- No `node_modules` or build artifacts included
- No `.env` files committed
- Comprehensive README with setup instructions
- Code is properly structured and documented
- Application runs without errors
- All features are functional

Demo & Documentation

- **Demo Video:** <https://drive.google.com/file/d/1SmWLG85v2LzqkVThGX0mt4uf1i5xPsnI/view>
- **Documentation:** Complete setup and usage instructions in README.md
- **API Documentation:** FastAPI auto-generated docs at `/docs`

Conclusion

This project demonstrates a complete, production-ready meeting summarization system with advanced features including real-time transcription, AI-powered analysis, and comprehensive user interface. All code follows best practices with proper error handling, type safety, and conventional commit messages.

Submitted by Akshat Sinha (22BCE2218)

Repository: https://github.com/akshatsinha0/AkshatSinha_22BCE2218_MeetingSummarizer